Selenium Automation Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Test Automation Engineer with 4+ years of experience

Questions in your original document	**955**
Exact duplicates removed (same question, different punctuation, case or formatting)	**91**
Near-duplicates merged (same question asked in different words)	**659**
Unique questions in this PDF	**205**

Section 1: Critical (Must Prepare)	65 questions
Section 2: High Priority	81 questions
Section 3: Medium Priority	48 questions
Section 4: Low Priority / Quick Revision	11 questions

How to Use This Document

Questions are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Prepare) 65 questions	Asked in almost every Selenium interview at 4+ years. Prepare a precise answer plus a real example from your own framework. Expect follow-ups such as 'what happened in your project?'.
High Priority 81 questions	Frequent practical and scenario questions. They check hands-on depth, so answer with the approach, the code idea and the problem it solves.
Medium Priority 48 questions	Occasionally asked, often as follow-ups or to probe a specific area from your resume. One clear answer each.
Low Priority / Quick Revision 11 questions	Niche or rarely asked at 4+ years. Skim them so you are never caught off guard.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
Selenium & WebDriver	Do you understand how WebDriver works (interface, architecture, drivers) rather than just using the API?
Locators, XPath & CSS	Choosing stable locators, writing XPath on the spot (axes, contains), and handling dynamic attributes.
Waits & Synchronization	Implicit vs explicit vs fluent, why mixing is risky, and how you actually remove Thread.sleep.
Element Interactions	Real project hands-on: dropdowns, Actions class, JavaScriptExecutor, scrolling, tables, screenshots.
Windows, Frames, Alerts	Switching contexts correctly and handling unexpected pop-ups.
Exceptions & Troubleshooting	How you diagnose Stale, NoSuchElement, ClickIntercepted errors, not just their names.
Flaky Tests & Debugging	Root-cause thinking: CI vs local, timing, data and environment issues, retry strategy.
POM & Page Factory	Clean design: why POM, how you structure pages, POM vs Page Factory, initialization.
Framework Design	Architecture ownership: layers, utilities, patterns, scalability. The most important section at 4+ years.
TestNG	Annotations and execution order, DataProvider, listeners, parallel runs and testng.xml configuration.
Cucumber & BDD	Feature files, step definitions, hooks, tags, runner, Scenario Outline and how it integrates with TestNG.
Data, Config, Reporting	Data-driven design, environment config, reports with screenshots, logging.
Cross-browser, Grid, CI/CD	Executing at scale: parallel runs, Grid/Docker, Jenkins pipelines, Maven and Git.
Scenario-Based	Judgement in real situations: what to automate, CAPTCHA/OTP, regression triage.
Java in Framework	Whether you can explain OOP and collections through your own framework code.
Strategy & Metrics	Senior-level thinking: ROI, estimates, metrics, maintenance and risks of over-automation.

Note: many entries were topic fragments (for example 'Scroll Until Element Visible' or 'Retry Failed Test Cases (TestNG)'); they are phrased as full questions and merged with matching questions. Questions on Java, Cucumber and TestNG that appear in several places are combined into one.

Table of Contents

Section 1: Critical (Must Prepare) (65 questions) . . . 5
Selenium & WebDriver Fundamentals (3) . . . 5
Locators, XPath & CSS (5) . . . 5
Waits & Synchronization (3) . . . 6
Element Interactions, Actions & JavaScriptExecutor (5) . . . 6
Windows, Frames, Alerts & Pop-ups (3) . . . 7
Exceptions & Troubleshooting (5) . . . 8
Flaky Tests, Retry & Debugging (6) . . . 9
File Upload/Download & Broken Links (3) . . . 9
Page Object Model & Page Factory (3) . . . 10
Framework Design & Architecture (4) . . . 11
TestNG (8) . . . 11
Cucumber & BDD (7) . . . 12
Test Data, Configuration, Reporting & Logging (4) . . . 14
Cross-browser, Grid, Docker, CI/CD, Maven & Git (2) . . . 14
Scenario-Based & Real-World Automation (2) . . . 15
Java Concepts in the Selenium Framework (2) . . . 15

Section 2: High Priority (81 questions) . . . 16
Selenium & WebDriver Fundamentals (7) . . . 16
Locators, XPath & CSS (5) . . . 17
Waits & Synchronization (5) . . . 17
Element Interactions, Actions & JavaScriptExecutor (11) . . . 18
Windows, Frames, Alerts & Pop-ups (4) . . . 20
Exceptions & Troubleshooting (1) . . . 20
Flaky Tests, Retry & Debugging (2) . . . 20
File Upload/Download & Broken Links (1) . . . 21
Page Object Model & Page Factory (2) . . . 21
Framework Design & Architecture (6) . . . 21
TestNG (7) . . . 22
Cucumber & BDD (10) . . . 23
Test Data, Configuration, Reporting & Logging (4) . . . 24
Cross-browser, Grid, Docker, CI/CD, Maven & Git (6) . . . 25
Scenario-Based & Real-World Automation (3) . . . 25
Java Concepts in the Selenium Framework (4) . . . 26
Automation Strategy, Metrics & Career (3) . . . 26

Section 3: Medium Priority (48 questions) . . . 27
Selenium & WebDriver Fundamentals (9) . . . 27
Locators, XPath & CSS (2) . . . 28
Waits & Synchronization (2) . . . 28
Element Interactions, Actions & JavaScriptExecutor (9) . . . 28
Windows, Frames, Alerts & Pop-ups (4) . . . 30
Exceptions & Troubleshooting (1) . . . 30
Flaky Tests, Retry & Debugging (1) . . . 30
File Upload/Download & Broken Links (1) . . . 30
Framework Design & Architecture (4) . . . 31
TestNG (1) . . . 31
Cucumber & BDD (4) . . . 31
Scenario-Based & Real-World Automation (5) . . . 32
Automation Strategy, Metrics & Career (5) . . . 33

Section 4: Low Priority / Quick Revision (11 questions) . . . 34
Selenium & WebDriver Fundamentals (1) . . . 34
Element Interactions, Actions & JavaScriptExecutor (3) . . . 34
TestNG (1) . . . 34
Cucumber & BDD (1) . . . 35
Scenario-Based & Real-World Automation (1) . . . 35
Automation Strategy, Metrics & Career (4) . . . 35

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	S4	Total
Selenium & WebDriver Fundamentals	3 Qs, p.5	7 Qs, p.16	9 Qs, p.27	1 Qs, p.34	**20**
Locators, XPath & CSS	5 Qs, p.5	5 Qs, p.17	2 Qs, p.28	-	**12**
Waits & Synchronization	3 Qs, p.6	5 Qs, p.17	2 Qs, p.28	-	**10**
Element Interactions, Actions & JavaScriptExecutor	5 Qs, p.6	11 Qs, p.18	9 Qs, p.28	3 Qs, p.34	**28**
Windows, Frames, Alerts & Pop-ups	3 Qs, p.7	4 Qs, p.20	4 Qs, p.30	-	**11**
Exceptions & Troubleshooting	5 Qs, p.8	1 Qs, p.20	1 Qs, p.30	-	**7**
Flaky Tests, Retry & Debugging	6 Qs, p.9	2 Qs, p.20	1 Qs, p.30	-	**9**
File Upload/Download & Broken Links	3 Qs, p.9	1 Qs, p.21	1 Qs, p.30	-	**5**
Page Object Model & Page Factory	3 Qs, p.10	2 Qs, p.21	-	-	**5**
Framework Design & Architecture	4 Qs, p.11	6 Qs, p.21	4 Qs, p.31	-	**14**
TestNG	8 Qs, p.11	7 Qs, p.22	1 Qs, p.31	1 Qs, p.34	**17**
Cucumber & BDD	7 Qs, p.12	10 Qs, p.23	4 Qs, p.31	1 Qs, p.35	**22**
Test Data, Configuration, Reporting & Logging	4 Qs, p.14	4 Qs, p.24	-	-	**8**
Cross-browser, Grid, Docker, CI/CD, Maven & Git	2 Qs, p.14	6 Qs, p.25	-	-	**8**
Scenario-Based & Real-World Automation	2 Qs, p.15	3 Qs, p.25	5 Qs, p.32	1 Qs, p.35	**11**
Java Concepts in the Selenium Framework	2 Qs, p.15	4 Qs, p.26	-	-	**6**
Automation Strategy, Metrics & Career	-	3 Qs, p.26	5 Qs, p.33	4 Qs, p.35	**12**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Prepare) (65 questions)

Asked in almost every Selenium interview at 4+ years. Prepare a precise answer plus a real example from your own framework. Expect follow-ups such as 'what happened in your project?'.

Selenium & WebDriver Fundamentals (3)

1. What is Selenium WebDriver and how does it work? Explain its architecture. **[asked x5 in your bank]**

   Answer: WebDriver is an API to automate browsers. Flow: test script (Java) > Selenium client library > W3C WebDriver protocol over HTTP > browser driver (chromedriver/geckodriver) > real browser.

   Example: Your login-to-checkout test sends commands to chromedriver, which performs the clicks and typing in Chrome.

2. What is the WebDriver interface? What does WebDriver driver = new ChromeDriver() mean? Which interfaces does Selenium use and where is dynamic polymorphism seen? **[asked x6 in your bank]**

   Answer: WebDriver is an interface; ChromeDriver is a class implementing it. Holding it as WebDriver gives dynamic polymorphism: the same code runs on any browser chosen at runtime. Other interfaces: WebElement, TakesScreenshot, JavascriptExecutor, Alert, TargetLocator.

   Example: driver = new ChromeDriver() or new FirefoxDriver() while the page classes use only WebDriver.

   ```
   WebDriver driver = new ChromeDriver();   // interface reference, class object
   ```

3. What is the difference between driver.close() and driver.quit()? **[asked x7 in your bank]**

   Answer: close() closes only the current window; quit() closes all windows of the session and ends the driver process.

   Example: After the checkout test, quit() in @AfterMethod to clean the session; close() only to shut a payment pop-up window.

   ```
   driver.close();   // current window only
   driver.quit();    // all windows + session
   ```

Locators, XPath & CSS (5)

4. What are the different types of locators in Selenium? Which do you prefer and what is their reliability order? **[asked x7 in your bank]**

   Answer: id, name, className, tagName, linkText, partialLinkText, cssSelector and xpath. Preferred order: id, name, CSS, then XPath; best of all is a dedicated test attribute such as data-testid.

   Example: Use By.id("add-to-cart") or By.cssSelector("[data-testid='add-to-cart']") instead of a long XPath.

5. What is the difference between findElement() and findElements()? What do they return and what happens when nothing is found? **[asked x13 in your bank]**

   Answer: findElement returns the first match and throws NoSuchElementException if none. findElements returns a List, which is empty when nothing is found (no exception).

   Example: Use findElements(By.id("cart-badge")).isEmpty() to check an empty cart without exceptions.

   ```
   boolean empty = driver.findElements(By.cssSelector(".cart-item")).isEmpty();
   ```

6. XPath vs CSS Selector: when do you use each, and which is faster? **[asked x8 in your bank]**

   Answer: CSS is generally faster, shorter and readable but cannot go to a parent or match text. XPath can traverse in any direction and use text(), at a slight speed cost. Use CSS by default, XPath when you need text or parent.

   Example: CSS: input#email. XPath: //button[text()='Place Order'] when only the text is unique.

7. What is XPath and what are its types? Difference between absolute and relative XPath. **[asked x7 in your bank]**

> **Answer:** XPath is a language to locate nodes in the DOM. Absolute XPath starts with / from the root (fragile); relative XPath starts with // and uses attributes (preferred).
>
> **Example:** Relative: //button[@id='checkout']. Absolute: /html/body/div[2]/div/button breaks when the layout changes.

8. How do you handle dynamic web elements (changing IDs or attributes)? Write a locator strategy. **[asked x12 in your bank]**

> **Answer:** Use stable parts of the attribute: starts-with, contains, ends-with, data attributes, or position relative to a stable label/parent. Avoid index-based locators.
>
> **Example:** Search box id changes like search_8392: use starts-with.

```
By.xpath("//input[starts-with(@id,'search_')]")
By.cssSelector("input[id^='search_']")
```

Waits & Synchronization (3)

9. What are the different waits in Selenium? Explain implicit vs explicit vs fluent wait and when to use each. **[asked x17 in your bank]**

> **Answer:** Implicit: global wait applied to every findElement. Explicit (WebDriverWait): waits for a specific condition on an element. Fluent: explicit wait with custom polling and ignored exceptions. Prefer explicit waits.
>
> **Example:** Wait for the Place Order button to be clickable instead of sleeping.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.elementToBeClickable(By.id("place-order"))).click();
```

10. Can implicit and explicit waits be mixed? Which one takes effect and is it good practice? **[asked x2 in your bank]**

> **Answer:** They can be mixed but it is bad practice: waits may add up and give unpredictable timeouts. Best practice is explicit waits with the implicit wait kept at 0.
>
> **Example:** A 10 s implicit plus a 10 s explicit wait can make a missing element take much longer to fail.

11. How do you handle AJAX, slow-loading pages or elements that appear late (dynamic waits, page load)? **[asked x9 in your bank]**

> **Answer:** Wait for conditions rather than time: visibility/clickability, spinner invisibility, URL/title, or document.readyState; set page load timeout for slow pages.
>
> **Example:** After clicking Apply Coupon, wait for the loader to disappear and the price to update.

```
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.cssSelector(".loader")));
wait.until(d -> ((JavascriptExecutor) d)
    .executeScript("return document.readyState").equals("complete"));
```

Element Interactions, Actions & JavaScriptExecutor (5)

12. How do you handle dropdowns (Select class: visible text, index, value; get all options; deselect)? **[asked x13 in your bank]**

> **Answer:** Wrap the select element in the Select class: selectByVisibleText, selectByValue, selectByIndex, getOptions, getFirstSelectedOption; deselect methods work only on multi-select.
>
> **Example:** Choose size "M" and read all sizes from the product size dropdown.

```
Select size = new Select(driver.findElement(By.id("size")));
size.selectByVisibleText("M");
List<WebElement> all = size.getOptions();
```

13. What is the Actions class and when do you use it? Actions vs direct WebElement methods. **[asked x6 in your bank]**

> **Answer:** Actions performs advanced user interactions: hover, right/double click, drag and drop, key combinations. Use it when plain click()/sendKeys() are not enough.
>
> **Example:** Hover over the Men menu and click the Shoes sub-menu.

```
new Actions(driver).moveToElement(menu).click(subMenu).perform();
```

14. What is JavaScriptExecutor and why/how is it used? **[asked x6 in your bank]**

> **Answer:** JavascriptExecutor runs JavaScript in the browser: scroll, click hidden/blocked elements, read or set values, get page state.
>
> **Example:** Click an Add to Cart button hidden behind a sticky banner.

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].click();", element);
```

15. How do you scroll a page, scroll to an element that is not in view, or scroll until the footer is visible? **[asked x9 in your bank]**

> **Answer:** Use JavaScript scrollBy, scrollIntoView, or scrollTo(0, document.body.scrollHeight) for the footer; Selenium 4 also has Actions.scrollToElement().
>
> **Example:** Scroll down to the footer of the product listing page.

```
js.executeScript("arguments[0].scrollIntoView(true);", element);
js.executeScript("window.scrollTo(0, document.body.scrollHeight);");
```

16. How do you capture a screenshot on test failure (named by test) and attach it to the report? **[asked x10 in your bank]**

> **Answer:** In an ITestListener's onTestFailure, cast the driver to TakesScreenshot, save the file named by test name and timestamp, and attach the path to the Extent/Allure report.
>
> **Example:** A failed checkout test automatically saves checkout_20261031.png in the report.

```
File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
Files.copy(src.toPath(), Paths.get("shots/" + result.getName() + ".png"));
```

Windows, Frames, Alerts & Pop-ups (3)

17. How do you handle alerts, pop-ups and browser dialogs (accept, dismiss, getText, sendKeys)? **[asked x18 in your bank]**

> **Answer:** Use driver.switchTo().alert() then accept(), dismiss(), getText() or sendKeys(). Wait with ExpectedConditions.alertIsPresent() first.
>
> **Example:** Accept the "Remove item from cart?" confirmation.

```
Alert alert = wait.until(ExpectedConditions.alertIsPresent());
String msg = alert.getText();
alert.accept();
```

18. How do you handle multiple windows or tabs and switch between them (parent window, title check)? **[asked x12 in your bank]**

> **Answer:** Store the parent handle, loop through getWindowHandles(), switch with switchTo().window(handle) and check the title or URL; switch back to the parent afterwards.
>
> **Example:** Payment opens in a new tab: switch, complete payment, return to the order page.

```
String parent = driver.getWindowHandle();
for (String h : driver.getWindowHandles())
  if (!h.equals(parent)) driver.switchTo().window(h);
driver.switchTo().window(parent);
```

19. How do you handle frames and iframes (switch in, switch back, nested frames)? **[asked x11 in your bank]**

> **Answer:** Use switchTo().frame(index/name/id/WebElement); come back with parentFrame() or defaultContent(). For nested frames switch step by step.
>
> **Example:** The card number field sits inside a payment iframe.

```
driver.switchTo().frame(driver.findElement(By.id("card-frame")));
driver.findElement(By.id("cardNumber")).sendKeys("4111111111111111");
driver.switchTo().defaultContent();
```

Exceptions & Troubleshooting (5)

20. What is StaleElementReferenceException? What causes it and how do you handle or avoid it (after refresh or AJAX)? **[asked x12 in your bank]**

> **Answer:** The element reference is no longer attached to the DOM (page refresh or AJAX re-render). Re-locate the element when needed, do not store WebElements long term, and use retry or ExpectedConditions.refreshed.
>
> **Example:** After applying a filter, the product list re-renders and the old product element becomes stale.

```
wait.until(ExpectedConditions.refreshed(
    ExpectedConditions.elementToBeClickable(By.cssSelector(".product"))));
```

21. What are the common exceptions you faced in Selenium and Java, and which was hardest to solve? **[asked x4 in your bank]**

> **Answer:** Common: NoSuchElement, StaleElementReference, TimeoutException, ElementClickIntercepted, ElementNotInteractable, NoSuchFrame, NullPointer and SessionNotCreated. Explain one you solved with the cause and fix.
>
> **Example:** Stale exceptions after the cart refreshed through AJAX, fixed with a fresh locator inside an explicit wait.

22. How do you avoid NoSuchElementException (with and without try/catch)? Intermittent NoSuchElement issues? **[asked x3 in your bank]**

> **Answer:** Use correct locators, explicit waits, check the frame/window context, and use findElements().size() for optional elements; use try-catch only when an element is genuinely optional.
>
> **Example:** The promo banner may not appear: check findElements before clicking Close.

23. The locator is correct and the element is on the page, but you still get "element not found". How do you resolve it? **[asked x2 in your bank]**

> **Answer:** Check if the element is inside an iframe or shadow DOM, in another window, loaded late (wait), duplicated, or loaded only after a scroll. Verify the locator in DevTools.
>
> **Example:** Review text lies inside an iframe: switch frame first and then locate it.

24. What causes ElementNotInteractable / ElementClickIntercepted exceptions, and what do you do when click() does not work? **[asked x4 in your bank]**

> **Answer:** NotInteractable: element hidden, disabled or off-screen. ClickIntercepted: another element (overlay, loader, sticky header) covers it. Fix: wait for the overlay to go, scroll into view, then use Actions or JavaScript click as a last resort.
>
> **Example:** A cookie banner covers Add to Cart: close the banner or wait for it to disappear first.

```
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.id("cookie-banner")));
wait.until(ExpectedConditions.elementToBeClickable(addToCart)).click();
```

Flaky Tests, Retry & Debugging (6)

25. How do you handle and avoid flaky tests? What are the common causes and how do you investigate them (CI, Jenkins, BrowserStack)? **[asked x13 in your bank]**

> **Answer:** Causes: poor synchronization, shared or changing data, test dependencies, environment instability, weak locators. Investigate by rerunning, reading logs/screenshots/video, comparing CI and local, then fix the root cause; quarantine truly flaky tests.
>
> **Example:** Payment test failed intermittently on Jenkins because the gateway responded slowly; fixed with a proper wait on the confirmation message.

26. How do you implement retry logic for failed tests (IRetryAnalyzer, retry a step in the same test)? **[asked x8 in your bank]**

> **Answer:** Implement IRetryAnalyzer.retry() with a counter and attach it via @Test(retryAnalyzer=...) or an annotation transformer listener. Keep retries small (1-2) and report them.
>
> **Example:** Retry once a payment test that fails because of a network blip, while still logging the first failure.

```
public class Retry implements IRetryAnalyzer {
  private int count = 0;
  public boolean retry(ITestResult r) { return count++ < 2; }
}
```

27. If a test fails, what is your next step? A script that worked yesterday fails today. How do you debug? **[asked x6 in your bank]**

> **Answer:** Read the error and screenshot, reproduce locally, decide whether the cause is an app change, locator, data, environment or timing, compare with the last passing build, then fix or raise a defect.
>
> **Example:** A test that passed yesterday fails at checkout: the build changed the button id; update the POM locator.

28. Tests pass locally but fail on Jenkins/CI nodes. How do you debug? **[asked x2 in your bank]**

> **Answer:** Differences in headless mode, screen size, speed, browser/driver versions, data and parallel runs. Set window size, use explicit waits, capture logs and screenshots on CI, and use the same browser/driver versions.
>
> **Example:** Menu hover works locally but the headless run has a smaller viewport so the menu collapses: set --window-size=1920,1080.

29. The application has minor changes (small CSS/HTML changes break many tests). How do you maintain and update scripts? **[asked x6 in your bank]**

> **Answer:** Keep locators in page objects (one place to fix), use stable locators such as data-testid, reusable components, and agree UI changes with developers early.
>
> **Example:** A class-name change on the Add to Cart button is fixed once in ProductPage instead of 40 tests.

30. Your automation scripts are slow, or the regression suite takes about 5 hours while stakeholders need results in under 1 hour. How do you speed it up? **[asked x3 in your bank]**

> **Answer:** Run in parallel (TestNG/Grid/Docker), headless mode, replace sleeps with waits, set up data through API, split suites (smoke/regression), keep tests independent and prune low-value tests.
>
> **Example:** Create cart and user data through API so UI tests start from the checkout page.

File Upload/Download & Broken Links (3)

31. How do you handle file upload in Selenium (sendKeys, Robot class, AutoIt)? Upload and download together? **[asked x8 in your bank]**

> **Answer:** For input type=file use sendKeys(absolutePath); no dialog is needed. If the input is hidden, unhide it with JavaScript. Robot/AutoIt are only for native OS dialogs.
>
> **Example:** Upload a profile picture or return-request image.

```
driver.findElement(By.cssSelector("input[type='file']"))
      .sendKeys("C:\\data\\invoice.pdf");
```

32. How do you handle file download and verify it (file exists, content is correct)? **[asked x8 in your bank]**

> **Answer:** Set the download folder in browser preferences, click the link, wait until the file exists (not .crdownload), then verify name, size and content.
>
> **Example:** Download the order invoice PDF and check it contains the order number.

```
Map<String, Object> prefs = new HashMap<>();
prefs.put("download.default_directory", downloadDir);
ChromeOptions opts = new ChromeOptions();
opts.setExperimentalOption("prefs", prefs);
// after click: wait until Files.exists(Paths.get(downloadDir, "invoice.pdf"))
```

33. How do you find broken links and broken images on a page? **[asked x11 in your bank]**

> **Answer:** Collect all <a> (or <img>) elements, send an HTTP request to each URL and flag status codes of 400 or more; for images also check naturalWidth equals 0.
>
> **Example:** Check all product and footer links on the home page return 200.

```
for (WebElement l : driver.findElements(By.tagName("a"))) {
  String url = l.getAttribute("href");
  HttpURLConnection c = (HttpURLConnection) new URL(url).openConnection();
  c.setRequestMethod("HEAD");
  if (c.getResponseCode() >= 400) System.out.println("Broken: " + url);
}
```

Page Object Model & Page Factory (3)

34. What is the Page Object Model (POM)? What are its advantages and how do you implement it (e.g. a LoginPage class)? **[asked x20 in your bank]**

> **Answer:** POM keeps each page as a class holding its locators and actions; tests call these methods. Advantages: reusability, easy maintenance, readable tests, less duplication.
>
> **Example:** LoginPage has login(user, pass); CartPage has addItem(); tests never use raw locators.

```
public class LoginPage {
  private WebDriver driver;
  private By email = By.id("email"), pwd = By.id("password");
  public LoginPage(WebDriver driver) { this.driver = driver; }
  public void login(String u, String p) {
    driver.findElement(email).sendKeys(u);
    driver.findElement(pwd).sendKeys(p);
  }
}
```

35. What is the difference between Page Object Model and Page Factory? **[asked x3 in your bank]**

> **Answer:** POM is a design pattern (pages as classes). Page Factory is one way to implement it using @FindBy and initElements. POM with By locators is the simpler, more common choice today.
>
> **Example:** LoginPage with By fields vs LoginPage with @FindBy annotated WebElements.

36. What is Page Factory? @FindBy vs driver.findElement, initElements and the annotations used in POM. **[asked x9 in your bank]**

> **Answer:** Page Factory initializes elements lazily from @FindBy annotations using PageFactory.initElements(driver, this). Related: @FindAll (OR), @FindBys (AND), @CacheLookup (caches, can cause stale elements).
>
> **Example:** Annotate the Search box and Search button in HomePage with @FindBy.

```
@FindBy(id = "search") private WebElement searchBox;
public HomePage(WebDriver driver) { PageFactory.initElements(driver, this); }
```

Framework Design & Architecture (4)

37. Explain your automation framework (architecture, folder structure and flow). **[asked x9 in your bank]**

> **Answer:** Describe layers: base (driver factory, config), pages (POM), tests, utilities (waits, Excel, screenshots), test data, listeners/reports, runners and CI. Flow: Jenkins > Maven > testng.xml > BaseTest > test > page methods > report.
>
> **Example:** E-commerce framework: pages for Login, Search, Cart, Checkout; tests per module; Extent report with screenshots.

38. How would you design a scalable, maintainable framework (Java + TestNG/Cucumber, UI and API, Docker, CI/CD) or build one from scratch? Best practices? **[asked x9 in your bank]**

> **Answer:** Layered design with POM, a ThreadLocal driver factory, environment-wise config, data-driven tests, API-assisted setup, logging/reporting, parallel runs on Grid/Docker, CI/CD, tags/groups, coding standards and code reviews.
>
> **Example:** Tests run on Docker Selenium Grid via Jenkins with smoke on every commit and regression nightly.

39. What is an automation framework, why do you need one, and what types exist (data-driven, keyword, hybrid, BDD)? **[asked x12 in your bank]**

> **Answer:** A framework is a structured set of guidelines, tools and reusable components for building and running tests. Types: modular, data-driven, keyword-driven, hybrid and BDD.
>
> **Example:** A hybrid framework: POM + TestNG + Excel/JSON data + Cucumber for business scenarios.

40. What was the most challenging thing in designing your framework, and what automation challenges did you overcome? **[asked x3 in your bank]**

> **Answer:** Pick a real challenge and show the result: thread-safe parallel execution, flaky tests, dynamic data, slow suites. Explain the problem, your approach and the measurable outcome.
>
> **Example:** Regression took 4 hours; with ThreadLocal drivers and Grid it dropped to 45 minutes.

TestNG (8)

41. What is TestNG and why use it? How does it differ from JUnit? **[asked x12 in your bank]**

> **Answer:** TestNG is a testing framework with annotations, groups, dependencies, DataProvider, listeners, parallel execution and XML suites. Compared with JUnit it has richer built-in suite and parallel control (JUnit 5 has narrowed the gap).
>
> **Example:** Group checkout tests as smoke and run them from testng.xml.

42. Explain TestNG annotations and their execution order (BeforeSuite, BeforeTest, BeforeClass, BeforeMethod). BeforeTest vs BeforeMethod. **[asked x15 in your bank]**

> **Answer:** Order: BeforeSuite, BeforeTest, BeforeClass, BeforeMethod, @Test, AfterMethod, AfterClass, AfterTest, AfterSuite. BeforeTest runs once per <test> tag; BeforeMethod runs before every @Test.
>
> **Example:** BeforeClass launches the browser; BeforeMethod opens the home page for each test.

43. How do priorities work in TestNG? How do you control execution order? **[asked x7 in your bank]**

> **Answer:** @Test(priority = n): lower number runs first, default 0, ties run alphabetically. For real dependencies use dependsOnMethods; keep tests independent where possible.
>
> **Example:** login (priority 1), addToCart (2), checkout (3).

```
@Test(priority = 1) public void login() { }
@Test(priority = 2) public void addToCart() { }
```

44. What is DataProvider and how do you use it for parameterization (and parameters through XML)? Attributes of @DataProvider. **[asked x13 in your bank]**

> **Answer:** A DataProvider returns Object[][] to run a test with multiple data sets. Attributes: name, parallel, indices. For values from XML use @Parameters and <parameter>.
>
> **Example:** Run the login test with valid, invalid and locked-account users.

```
@DataProvider(name = "users")
public Object[][] users() {
  return new Object[][] {{"user1", "pass1"}, {"user2", "bad"}};
}
@Test(dataProvider = "users")
public void login(String u, String p) { }
```

45. What is testng.xml used for? How do you configure and run tests, add or remove tests, and run multiple suites? **[asked x11 in your bank]**

> **Answer:** testng.xml defines suites, tests, classes, packages, groups, parameters, listeners and parallel settings. Run it from the IDE, Maven Surefire or command line; include/exclude to add/remove tests and use <suite-files> for several suites.
>
> **Example:** Smoke.xml runs only login, search and checkout classes.

```
<suite name="Regression" parallel="classes" thread-count="3">
  <test name="Checkout"><classes><class name="tests.CheckoutTest"/></classes></test>
</suite>
```

46. How do you run tests in parallel in TestNG (parallel attribute, testng.xml or Maven) and make the framework thread-safe? **[asked x17 in your bank]**

> **Answer:** Set parallel="methods|classes|tests" and thread-count in testng.xml (or Surefire config). Make the framework thread-safe using ThreadLocal<WebDriver>, no shared static state, and independent test data.
>
> **Example:** Run the cart and payment classes in parallel with separate browsers.

```
private static ThreadLocal<WebDriver> tl = new ThreadLocal<>();
public static WebDriver getDriver() { return tl.get(); }
```

47. How do you execute only the failed test cases (e.g. 50 tests, 5 failed) in TestNG? **[asked x4 in your bank]**

> **Answer:** After a run TestNG creates testng-failed.xml in test-output; run it to rerun only failures. Alternatives: IRetryAnalyzer, Surefire rerunFailingTestsCount or a CI rerun job.
>
> **Example:** 50 tests ran, 5 failed: execute testng-failed.xml to rerun the 5.

48. Hard vs soft assertions and Assert vs Verify: when do you use which? **[asked x10 in your bank]**

> **Answer:** Hard assertion (Assert) stops the test on the first failure. Soft assertion (SoftAssert) records failures and reports them at assertAll(). Use hard for preconditions, soft for several independent checks on one page.
>
> **Example:** On the product page, soft-assert title, price and image together; hard-assert login success.

```
SoftAssert sa = new SoftAssert();
sa.assertEquals(title, "Blue Shirt");
sa.assertEquals(price, "999");
sa.assertAll();
```

Cucumber & BDD (7)

49. What is BDD and Cucumber? Why is it used with Selenium? Explain Gherkin syntax and keywords. Have you used it? **[asked x14 in your bank]**

> **Answer:** BDD describes behaviour in plain English (Gherkin) that business and testers share. Cucumber runs these feature files by mapping steps to Java. Keywords: Feature, Scenario, Given, When, Then, And, But, Background, Scenario Outline, Examples.
>
> **Example:** Business-readable checkout scenarios linked to Selenium step definitions.

```
Feature: Checkout
  Scenario: Place order with a valid card
    Given the user has a product in the cart
    When the user pays with a valid card
    Then the order confirmation is displayed
```

50. How are step definitions created and mapped to feature file steps? What if multiple definitions match the same step? **[asked x6 in your bank]**

> **Answer:** Each Gherkin step maps to a Java method annotated with @Given/@When/@Then using a Cucumber expression or regex, found through the glue package. If more than one definition matches, Cucumber throws AmbiguousStepDefinitionsException, so make expressions unique.
>
> **Example:** "When the user adds {string} to the cart" maps to one method.

```
@When("the user adds {string} to the cart")
public void addToCart(String product) { cartPage.add(product); }
```

51. What are hooks in Cucumber (@Before, @After)? Hooks vs tags. **[asked x9 in your bank]**

> **Answer:** Hooks are code blocks (@Before, @After, @BeforeStep, @AfterStep) that run around scenarios for setup and teardown. Tags are labels used to select or filter scenarios; a hook can be limited to a tag.
>
> **Example:** @Before launches the browser; @After("@ui") quits it and takes a screenshot on failure.

```
@Before public void setUp() { driver = DriverFactory.create(); }
@After public void tearDown(Scenario s) {
  if (s.isFailed()) s.attach(screenshot(), "image/png", "failure");
  driver.quit();
}
```

52. What are tags in Cucumber? How do you run specific scenarios or combinations of tags? **[asked x8 in your bank]**

> **Answer:** Tags such as @smoke or @regression are placed above a Feature or Scenario. Run them using the runner tags option with and/or/not expressions.
>
> **Example:** Run only smoke checkout scenarios and skip work in progress.

```
@CucumberOptions(tags = "@smoke and not @wip")
```

53. What is the difference between Scenario and Scenario Outline? Explain Examples. **[asked x5 in your bank]**

> **Answer:** Scenario runs once. Scenario Outline runs repeatedly for each row in the Examples table, using <placeholders>.
>
> **Example:** Login with several user and password combinations from one outline.

```
Scenario Outline: Login
  When the user logs in with "<user>" and "<pass>"
  Then the message "<result>" is shown
  Examples:
    | user  | pass | result  |
    | user1 | pwd1 | Welcome |
    | user1 | bad  | Invalid |
```

54. What is the Cucumber runner class and how is it configured (glue, features, plugins)? Write the skeleton. **[asked x8 in your bank]**

> **Answer:** The runner class connects Cucumber with the test engine: feature location, glue (step/hook packages), plugins (reports) and tags. With TestNG it extends AbstractTestNGCucumberTests.
>
> **Example:** One runner for smoke and one for regression using different tags.

```
@CucumberOptions(features = "src/test/resources/features", glue = {"stepdefs", "hooks"},
    plugin = {"pretty", "html:target/cucumber.html"}, tags = "@smoke")
public class RunnerTest extends AbstractTestNGCucumberTests { }
```

55. How do you integrate Cucumber with Selenium WebDriver and TestNG? **[asked x4 in your bank]**

> **Answer:** Hooks create and quit the driver, step definitions call page objects, and the runner extends AbstractTestNGCucumberTests. Share the driver between classes with dependency injection (PicoContainer) or a context class.
>
> **Example:** Steps for Search, Cart and Payment reuse one driver through a TestContext object.

Test Data, Configuration, Reporting & Logging (4)

56. How do you implement data-driven testing and manage test data in your framework (Excel, CSV, JSON)? **[asked x13 in your bank]**

> **Answer:** Keep data outside the code (Excel, CSV, JSON), read it through a utility and feed tests via TestNG DataProvider or Cucumber Examples. Separate data per environment and keep it independent per test.
>
> **Example:** Login credentials and product SKUs stored in JSON and read by DataProvider.

57. How do you read data from an Excel file in Java? Write the code. **[asked x4 in your bank]**

> **Answer:** Use Apache POI: open the file, get the sheet, loop over rows and cells and read values (use DataFormatter to read any type as text).
>
> **Example:** Read username/password rows for the login DataProvider.

```
try (Workbook wb = WorkbookFactory.create(new File("data/users.xlsx"))) {
  Sheet sheet = wb.getSheet("Login");
  DataFormatter fmt = new DataFormatter();
  for (Row row : sheet)
    System.out.println(fmt.formatCellValue(row.getCell(0)));
}
```

58. How do you manage environment configuration (URLs, credentials, toggles for DEV/QA/UAT/PROD) and config.properties? **[asked x10 in your bank]**

> **Answer:** Store URLs, credentials and toggles in config.properties per environment (dev/qa/uat) and select with -Denv=qa. Read through a ConfigReader class; keep secrets in CI variables.
>
> **Example:** qa.properties holds the QA shop URL; the pipeline passes -Denv=uat for the UAT run.

```
String env = System.getProperty("env", "qa");
Properties p = new Properties();
p.load(new FileInputStream("config/" + env + ".properties"));
String url = p.getProperty("base.url");
```

59. How do you generate reports (Extent/Allure) with screenshots, steps and logs? Where are reports stored or published? **[asked x14 in your bank]**

> **Answer:** Use ExtentReports or Allure: a listener logs steps and attaches a screenshot on failure; the HTML report is stored under target/ and published as a Jenkins artifact or via the HTML Publisher plugin.
>
> **Example:** Nightly regression report with failing checkout screenshots is emailed to the team.

Cross-browser, Grid, Docker, CI/CD, Maven & Git (2)

60. How do you implement cross-browser testing and run across browsers, versions and environments? **[asked x8 in your bank]**

> **Answer:** Create a driver factory that returns the driver based on a browser parameter (testng.xml/CLI); use Grid or a cloud provider (BrowserStack) for different browsers and versions.
>
> **Example:** Run the checkout suite on Chrome, Firefox and Edge from one suite file.

```
public static WebDriver create(String browser) {
  switch (browser.toLowerCase()) {
    case "firefox": return new FirefoxDriver();
    case "edge":    return new EdgeDriver();
    default:        return new ChromeDriver();
  }
}
```

61. Explain CI/CD and how you integrate automation with Jenkins or Azure DevOps. **[asked x5 in your bank]**

> **Answer:** CI builds and tests code on every change; CD deploys it. A Jenkins/Azure DevOps pipeline checks out Git, runs mvn clean test with the environment, publishes reports and notifies the team.
>
> **Example:** A commit to the cart service triggers the smoke suite; nightly job runs full regression.

```
pipeline {
  agent any
  stages {
    stage('Test') { steps { sh 'mvn clean test -Denv=qa -DsuiteXmlFile=smoke.xml' } }
  }
  post { always { publishHTML(target: [reportDir: 'target', reportFiles: 'report.html',
      reportName: 'Test Report']) } }
}
```

Scenario-Based & Real-World Automation (2)

62. How would you automate login (including multiple user roles and permissions) and handle authentication? **[asked x5 in your bank]**

> **Answer:** Data-drive credentials and roles, log in through a page object, verify role-specific menus and restrictions; for speed log in via API/cookies and reuse the session in other tests. Cover negative cases (wrong password, locked account).
>
> **Example:** Customer, seller and admin roles each see different menus after login.

63. How do you decide what to automate and what not to automate? (e.g. a long-running scenario: manual or automation?) **[asked x4 in your bank]**

> **Answer:** Automate stable, repetitive, high-risk and data-heavy scenarios (regression, smoke, cross-browser). Do not automate one-off, frequently changing, exploratory, usability or CAPTCHA cases. A long scenario is automated if it is repeated and stable (run it nightly), otherwise left manual.
>
> **Example:** Automate login, search, cart and checkout; keep a new unstable promo-banner design manual until it settles.

Java Concepts in the Selenium Framework (2)

64. Explain OOP concepts and where you applied them in your framework (inheritance, polymorphism, abstraction, encapsulation). **[asked x6 in your bank]**

> **Answer:** Inheritance: BaseTest/BasePage. Polymorphism: WebDriver reference to different drivers. Abstraction: interfaces/abstract classes for pages or utilities. Encapsulation: private locators with public methods.
>
> **Example:** CartPage extends BasePage; locators private; WebDriver driver = new ChromeDriver().

65. Overloading vs overriding in a Selenium framework, with examples. **[asked x3 in your bank]**

> **Answer:** Overloading: same method name, different parameters (compile time). Overriding: child class redefines a parent method (runtime). Selenium itself overloads switchTo().frame(index/name/element).
>
> **Example:** click(By) and click(By, int timeout) in a utility class (overloading); a ProductPage overriding BasePage.isLoaded() (overriding).

```
void click(By by) { }
void click(By by, int timeoutSeconds) { }   // overloading
```

Section 2: High Priority (81 questions)

Frequent practical and scenario questions. They check hands-on depth, so answer with the approach, the code idea and the problem it solves.

Selenium & WebDriver Fundamentals (7)

66. What is Selenium and what are its core components (IDE, RC, WebDriver, Grid)? **[asked x4 in your bank]**

> **Answer:** Selenium is an open-source browser automation suite: IDE (record/playback), RC (old, removed), WebDriver (main API) and Grid (parallel/remote execution).
>
> **Example:** Most teams use WebDriver with Grid for cross-browser regression of the shop.

67. What are the limitations of Selenium? **[asked x3 in your bank]**

> **Answer:** Web applications only (mobile needs Appium), no built-in reporting or test management, cannot handle CAPTCHA/OTP, limited desktop/image/PDF support, and synchronization needs care.
>
> **Example:** Selenium cannot read the OTP sent by SMS, so the OTP step needs a test bypass or API.

68. Selenium vs Playwright: which is better and when? Can the same locators be reused? **[asked x2 in your bank]**

> **Answer:** Playwright has auto-waiting, built-in tracing and faster setup; Selenium is mature with many languages, a huge ecosystem and Grid. CSS/XPath strings can be reused but the APIs differ, so tests must be rewritten. Choose based on team skills, browser needs and existing investment.
>
> **Example:** Keep the existing Selenium regression suite and try Playwright for a new web module.

69. What is the difference between driver.get(), driver.navigate().to() and driver.switchTo()? What are the navigation commands? **[asked x5 in your bank]**

> **Answer:** get() and navigate().to() both open a URL; navigate() also keeps history with back(), forward() and refresh(). switchTo() is not navigation: it changes focus to frames, windows or alerts.
>
> **Example:** navigate().back() from checkout to the cart page.

```
driver.get("https://shop.example.com");
driver.navigate().back();
driver.navigate().refresh();
```

70. What do getText(), getAttribute(), getDomAttribute() and getDomProperty() do, and how do they differ? **[asked x4 in your bank]**

> **Answer:** getText() returns visible text. getDomAttribute() returns the attribute as written in the HTML; getDomProperty() returns the live JavaScript property (for example a typed input value); getAttribute() returns the property if present, otherwise the attribute.
>
> **Example:** After typing in the search box, getDomProperty("value") returns the typed text while getDomAttribute("value") returns the original HTML value.

```
String typed = box.getDomProperty("value");
String initial = box.getDomAttribute("value");
```

71. How do you check whether an element is displayed, enabled or selected? How do you check a disabled button without isEnabled()? **[asked x6 in your bank]**

> **Answer:** Use isDisplayed(), isEnabled() and isSelected(). For a disabled button without isEnabled(), check the disabled attribute, an aria-disabled value or a CSS class.
>
> **Example:** Place Order must be disabled until the terms checkbox is selected.

```
boolean disabled = btn.getDomAttribute("disabled") != null
    || "true".equals(btn.getDomAttribute("aria-disabled"));
```

72. What ChromeOptions have you used and why?

> **Answer:** Common options: --headless=new, --window-size, --start-maximized, --disable-notifications, --incognito, acceptInsecureCerts and download/pop-up preferences.
>
> **Example:** Headless with a fixed 1920x1080 window for stable CI runs.

```
ChromeOptions o = new ChromeOptions();
o.addArguments("--headless=new", "--window-size=1920,1080", "--disable-notifications");
WebDriver driver = new ChromeDriver(o);
```

Locators, XPath & CSS (5)

73. Explain XPath functions and axes: contains(), starts-with(), normalize-space(), following-sibling, preceding and the :: operator. **[asked x4 in your bank]**

> **Answer:** contains(), starts-with() and normalize-space() match partial/trimmed text or attributes. Axes (following-sibling, preceding, parent, ancestor) move across the tree; :: separates the axis from the node.
>
> **Example:** Find the price next to a product name.

```
//span[contains(@class,'price')]
//h2[normalize-space()='Blue Shirt']/following-sibling::span[1]
//input[@id='qty']/parent::div
```

74. What is the difference between "/" and "//" in XPath? Write XPath for an element with specific text or from given HTML. **[asked x4 in your bank]**

> **Answer:** / selects a direct child (or starts at the root); // selects descendants anywhere. Text lookups use text() or contains(text(), ...).
>
> **Example:** //div[@class='cart']//button[text()='Remove'] finds Remove anywhere inside the cart div.

75. How do you select the nth (e.g. third) element among multiple similar elements using XPath and CSS? **[asked x4 in your bank]**

> **Answer:** XPath uses (//li)[3] or //ul/li[3]; CSS uses li:nth-child(3) or li:nth-of-type(3).
>
> **Example:** Select the third product card in the listing.

```
By.xpath("(//div[@class='product'])[3]")
By.cssSelector(".product:nth-of-type(3)")
```

76. How do you handle shadow DOM elements (and iframe switching)? **[asked x4 in your bank]**

> **Answer:** Selenium 4: get the host element, call getShadowRoot() and search inside it with CSS (XPath is not supported inside shadow roots). iframes are separate and need switchTo().frame().
>
> **Example:** A chat widget built as a web component is reached through its shadow root.

```
SearchContext root = driver.findElement(By.cssSelector("chat-widget")).getShadowRoot();
root.findElement(By.cssSelector("button.send")).click();
```

77. What is a WebElement and how do you locate WebElements?

> **Answer:** WebElement is the interface for an HTML element; locate it with findElement(By...). Its methods include click, sendKeys, getText, getAttribute, isDisplayed and isEnabled.
>
> **Example:** WebElement addBtn = driver.findElement(By.id("add-to-cart"));

Waits & Synchronization (5)

78. What is synchronization in Selenium? **[asked x3 in your bank]**

> **Answer:** Synchronization aligns script speed with application speed using waits so actions happen when the element is ready.
>
> **Example:** Wait for cart count to update before checking it.

79. Pros and cons of implicit and explicit waits. Why prefer explicit over fluent? What is the implicit wait best practice? **[asked x4 in your bank]**

> **Answer:** Implicit: simple and global but hides timing issues and cannot wait for conditions. Explicit: waits for specific conditions, more control, more code. Fluent is explicit with polling/ignore options (WebDriverWait is built on it). Best practice: explicit waits, implicit at 0.
>
> **Example:** Explicit wait for the order confirmation instead of a global implicit wait.

80. What is the default polling time of explicit/fluent wait? **[asked x2 in your bank]**

> **Answer:** Both explicit (WebDriverWait) and fluent waits poll every 500 milliseconds by default.
>
> **Example:** A 10 s wait checks the condition about every half a second until it is true.

81. What are Expected Conditions? Give 5 examples. How do urlToBe() and titleIs() differ? **[asked x3 in your bank]**

> **Answer:** ExpectedConditions are ready-made wait conditions: visibilityOfElementLocated, elementToBeClickable, presenceOfElementLocated, alertIsPresent, invisibilityOfElementLocated, titleIs, urlContains. urlToBe needs the exact URL; titleIs needs the exact title.
>
> **Example:** wait.until(ExpectedConditions.urlContains("/order-success")) after payment.

82. Write code for implicit, explicit and fluent waits, and a custom FluentWait instead of Thread.sleep. Thread.sleep vs implicit wait? **[asked x7 in your bank]**

> **Answer:** Implicit: driver.manage().timeouts().implicitlyWait(); explicit: WebDriverWait; fluent: FluentWait with polling and ignoring. Thread.sleep always waits the full time, while waits return as soon as the condition is met.
>
> **Example:** Fluent wait that ignores NoSuchElementException while waiting for the cart total.

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(0));
Wait<WebDriver> fw = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(20)).pollingEvery(Duration.ofMillis(500))
    .ignoring(NoSuchElementException.class);
WebElement total = fw.until(d -> d.findElement(By.id("cart-total")));
```

Element Interactions, Actions & JavaScriptExecutor (11)

83. How do you handle dynamic dropdowns (values change by previous selection, 500 values, last option, Streams)? **[asked x4 in your bank]**

> **Answer:** For custom dropdowns click to open, wait for options, then pick by text with a stream or loop; the last option is options.get(size - 1); for very long lists filter by typing or scroll.
>
> **Example:** Choose a city from a 500-city delivery dropdown by typing and selecting.

```
List<WebElement> opts = driver.findElements(By.cssSelector(".dropdown li"));
opts.stream().filter(o -> o.getText().equals("Pune")).findFirst().get().click();
```

84. How do you handle auto-suggest / auto-fill fields and validate the listed options? **[asked x2 in your bank]**

> **Answer:** Type part of the text, wait for suggestions, collect their text to validate and click the required one.
>
> **Example:** Search for "shoe" and verify all suggestions contain "shoe".

```
box.sendKeys("shoe");
List<WebElement> s = wait.until(
    ExpectedConditions.visibilityOfAllElementsLocatedBy(By.cssSelector(".suggest li")));
for (WebElement e : s) Assert.assertTrue(e.getText().toLowerCase().contains("shoe"));
```

85. How do you perform mouse hover, double-click and right-click? **[asked x6 in your bank]**

> **Answer:** Use the Actions class: moveToElement for hover, doubleClick and contextClick for the others.
>
> **Example:** Hover on a product image to show Quick View, right-click the image.

```
Actions act = new Actions(driver);
act.moveToElement(img).perform();
act.doubleClick(el).perform();
act.contextClick(el).perform();
```

86. How do you automate drag and drop? **[asked x3 in your bank]**

> **Answer:** Use Actions.dragAndDrop(source, target); if HTML5 drag/drop fails, use clickAndHold, moveToElement, release, or a JavaScript approach.
>
> **Example:** Drag a product into the cart area or reorder wishlist items.

```
new Actions(driver).dragAndDrop(source, target).perform();
```

87. How do you handle keyboard events (ENTER key, keyboard shortcuts)? **[asked x3 in your bank]**

> **Answer:** Use sendKeys(Keys.X) for single keys and Actions keyDown/keyUp for combinations.
>
> **Example:** Press ENTER to submit the search; Ctrl+A to select the text in a box.

```
box.sendKeys("shoes", Keys.ENTER);
new Actions(driver).keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform();
```

88. Why do we use build() and perform() with Actions? Can perform() be used without build()? **[asked x3 in your bank]**

> **Answer:** build() creates the combined action; perform() executes it. perform() calls build() internally, so build() is only needed when you store the action to run later.
>
> **Example:** Chain moveToElement and click, then call perform().

89. How do you handle hidden or invisible elements? **[asked x5 in your bank]**

> **Answer:** Check why it is hidden: wait for it to appear or trigger it; to read hidden text use JavaScript or getDomProperty("textContent"); use JavaScript click only when the user could not otherwise reach it.
>
> **Example:** Read the hidden SKU text in the product page.

```
String sku = (String) ((JavascriptExecutor) driver)
    .executeScript("return arguments[0].textContent;", hiddenEl);
```

90. How do you handle web tables (dynamic data, extract data, verify sorting)? **[asked x4 in your bank]**

> **Answer:** Locate rows (//table//tr) and cells (td[n]), loop to read or compare values, handle pagination, and verify sorting by comparing the extracted list with a sorted copy.
>
> **Example:** Verify the Orders table is sorted by price ascending.

```
List<Double> prices = new ArrayList<>();
for (WebElement c : driver.findElements(By.xpath("//table//tr/td[3]")))
  prices.add(Double.parseDouble(c.getText()));
List<Double> sorted = new ArrayList<>(prices);
Collections.sort(sorted);
Assert.assertEquals(prices, sorted);
```

91. How do you handle date pickers / calendars? **[asked x2 in your bank]**

> **Answer:** If the field accepts typing use sendKeys(date); otherwise open the picker and click the day, month and year dynamically (calculate the target date) or set the value with JavaScript.
>
> **Example:** Pick a delivery date 3 days from today in the calendar widget.

```
String day = String.valueOf(LocalDate.now().plusDays(3).getDayOfMonth());
driver.findElement(By.xpath("//td[@data-day='" + day + "']")).click();
```

92. How do you handle SSL certificate errors? **[asked x6 in your bank]**

> **Answer:** Set acceptInsecureCerts to true in the browser options.
>
> **Example:** Test environment with a self-signed certificate.

```
ChromeOptions o = new ChromeOptions();
o.setAcceptInsecureCerts(true);
```

93. How do you take a screenshot? Is TakesScreenshot an interface or a class? **[asked x6 in your bank]**

> **Answer:** TakesScreenshot is an interface. Cast the driver to it, call getScreenshotAs(OutputType.FILE) and copy the file; for an element, call getScreenshotAs on the WebElement.
>
> **Example:** Save a screenshot of the order confirmation page.

```
File f = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(f, new File("shots/confirmation.png"));
```

Windows, Frames, Alerts & Pop-ups (4)

94. What are the return types of getWindowHandle() and getWindowHandles()? **[asked x3 in your bank]**

> **Answer:** getWindowHandle() returns a String (current window); getWindowHandles() returns a Set<String> of all window IDs.
>
> **Example:** Loop through the set after the payment pop-up opens.

```
String current = driver.getWindowHandle();
Set<String> all = driver.getWindowHandles();
```

95. Pop-ups appear at unpredictable times. How do you handle them with WebDriver? **[asked x2 in your bank]**

> **Answer:** Create a helper that checks for the pop-up with findElements (or a short wait) before main actions and closes it if present; disable browser notifications through options.
>
> **Example:** A newsletter pop-up appears randomly: a closeIfPresent() helper runs before each page action.

```
List<WebElement> popup = driver.findElements(By.id("newsletter-close"));
if (!popup.isEmpty() && popup.get(0).isDisplayed()) popup.get(0).click();
```

96. How do you validate data inside a modal or pop-up?

> **Answer:** Wait for the modal to be visible, locate elements inside the modal container, assert text/values, then close it and make sure the page behind is active again.
>
> **Example:** Validate the size-chart modal values and close it.

```
WebElement modal = wait.until(
    ExpectedConditions.visibilityOfElementLocated(By.id("size-modal")));
String text = modal.findElement(By.cssSelector(".title")).getText();
```

97. How do you handle Windows-based pop-ups and Windows applications (AutoIt, Robot)? **[asked x4 in your bank]**

> **Answer:** Selenium cannot control native OS dialogs. Use Robot class (keyboard events), AutoIt/Sikuli, or tools like Appium Windows driver; better still avoid the dialog by sending the file path directly.
>
> **Example:** A native print dialog from Print Invoice is handled with Robot or skipped.

Exceptions & Troubleshooting (1)

98. What is your strategy when Selenium cannot automate a specific step?

> **Answer:** Use alternatives: JavaScriptExecutor, an API call for the same step, DevTools, Robot/AutoIt, a different tool for that part, or cover that step manually and document it.
>
> **Example:** OTP step in checkout is bypassed with a test API while the rest stays in Selenium.

Flaky Tests, Retry & Debugging (2)

99. A regression release is due tonight and overnight CI shows failures in modules that were not touched. How do you triage?

> **Answer:** Check environment and build first, group failures by root cause, rerun the failed tests, compare with the last green run and recent commits, separate real regressions from infrastructure/flaky issues, then report risk and a go/no-go with evidence.
>
> **Example:** 20 failures in Wishlist after a payment-only change point to an environment or data problem, not a code regression.

100. How do you deal with unstable or inconsistent test environments?

> **Answer:** Run a health check before the suite, keep tests independent, use API setup, retry only infrastructure errors, report environment failures separately and work with DevOps for a stable environment.
>
> **Example:** If the QA payment sandbox is down, mark those tests as environment failures, not product bugs.

File Upload/Download & Broken Links (1)

101. How do you handle file upload when the element is not type=file, or in remote execution (Grid/cloud)? **[asked x2 in your bank]**

> **Answer:** Find the hidden input type=file in the DOM and send the path to it (unhide with JavaScript if needed). For remote execution set a LocalFileDetector.
>
> **Example:** Custom Upload Image button for a return request.

```
RemoteWebDriver rd = (RemoteWebDriver) driver;
rd.setFileDetector(new LocalFileDetector());
rd.findElement(By.cssSelector("input[type='file']")).sendKeys("/data/photo.png");
```

Page Object Model & Page Factory (2)

102. How do you initialize web elements in POM? What exception occurs if you do not? What do you write first in a page class? **[asked x3 in your bank]**

> **Answer:** Call PageFactory.initElements(driver, this) in the page constructor (the first statement after assigning the driver); without it a NullPointerException occurs when the @FindBy element is used.
>
> **Example:** LoginPage constructor initializes the @FindBy fields.

```
public LoginPage(WebDriver driver) {
  this.driver = driver;
  PageFactory.initElements(driver, this);
}
```

103. You have 10 pages. How do you achieve POM, call page methods from tests and use it with TestNG or Cucumber? **[asked x4 in your bank]**

> **Answer:** Create one class per page; tests create page objects and call their methods (methods can return the next page object). Use the same page classes from TestNG tests or Cucumber step definitions with a shared driver.
>
> **Example:** HomePage.search() returns ProductListPage, which returns CartPage.

Framework Design & Architecture (6)

104. What is a hybrid framework and how do you create one? **[asked x2 in your bank]**

> **Answer:** A hybrid framework combines approaches: POM plus data-driven (Excel/JSON), TestNG, reports and optionally BDD/keyword-driven. Build it in layers with shared utilities and config.
>
> **Example:** POM + TestNG + JSON data + Extent report for the shop.

105. Which do you prefer: a Cucumber or a TestNG hybrid framework? Why?

> **Answer:** Cucumber suits teams needing business-readable scenarios and collaboration but adds a layer and maintenance. TestNG hybrid is simpler and faster for technical teams. Choose based on stakeholders and team skills.
>
> **Example:** If product owners review scenarios, use Cucumber; otherwise TestNG is enough.

106. What are the essential components of an automation framework?

> **Answer:** Driver management, configuration, page objects, utilities (waits, JS, data), test data handling, reporting, logging, listeners/retry, test runner/CI integration and version control.
>
> **Example:** DriverFactory, ConfigReader, pages, ExcelUtil, ExtentManager and a Jenkinsfile.

107. Which design patterns have you used (Singleton, Factory, POM)? Where do you use Singleton? Factory vs Singleton? **[asked x4 in your bank]**

> **Answer:** Used patterns: Page Object, Singleton (single config reader/report manager), Factory (creates driver by browser), Builder (test data). Singleton returns one shared object; Factory creates the right object for an input.
>
> **Example:** DriverFactory returns Chrome/Firefox; ConfigReader is a singleton.

108. How do you build reusable components, utility classes and modular tests? **[asked x4 in your bank]**

> **Answer:** Create utility classes (WaitUtil, JsUtil, ExcelUtil), base classes, reusable page components (header, footer) and small modular test methods that can be combined.
>
> **Example:** A HeaderComponent used by every page for search and cart icon actions.

109. How do you integrate API and database testing with UI automation? API vs UI frameworks? **[asked x3 in your bank]**

> **Answer:** Use API/DB for test setup and validation and UI for end-to-end flows. Example: create the user and cart by API, verify the order in the UI, then confirm it by API/DB query. API tests are faster and more stable than UI tests.
>
> **Example:** Create order via REST API, then check the order appears in My Orders on the UI.

TestNG (7)

110. What are TestNG Listeners (ITestListener, IReporter)? What types exist and how do you implement a custom listener? **[asked x7 in your bank]**

> **Answer:** TestNG listeners observe test events: ITestListener (start/success/failure/skip), ISuiteListener, IReporter, IInvokedMethodListener and IAnnotationTransformer. Implement the interface and register with @Listeners or testng.xml.
>
> **Example:** A listener captures a screenshot and logs to Extent on every failure.

```
public class TestListener implements ITestListener {
  @Override public void onTestFailure(ITestResult r) { Screenshot.take(r.getName()); }
}
```

111. How do you group test cases and run specific groups in TestNG? **[asked x6 in your bank]**

> **Answer:** Add groups to @Test and select them in testng.xml with include/exclude, or from Maven with -Dgroups.
>
> **Example:** Run only smoke-tagged checkout tests before every deployment.

```
@Test(groups = {"smoke", "checkout"})
public void placeOrder() { }
// testng.xml: <groups><run><include name="smoke"/></run></groups>
```

112. How do you create dependencies between tests (dependsOnMethods)? **[asked x4 in your bank]**

> **Answer:** Use @Test(dependsOnMethods="login"). If the dependency fails, the dependent test is skipped; alwaysRun=true overrides that.
>
> **Example:** addToCart depends on login, so it is skipped if login fails.

```
@Test public void login() { }
@Test(dependsOnMethods = "login") public void addToCart() { }
```

113. How do you skip a test in TestNG (enabled=false, SkipException)? **[asked x5 in your bank]**

> **Answer:** Use @Test(enabled = false), throw new SkipException("reason") at run time, or exclude it in testng.xml.
>
> **Example:** Skip the Apple Pay test when the environment has no wallet configured.

```
if (!wallet.isAvailable()) throw new SkipException("Wallet not available");
```

114. What are the @Test attributes (invocationCount, timeOut, expectedExceptions, groups, alwaysRun)? **[asked x5 in your bank]**

> **Answer:** Common attributes: priority, groups, dependsOnMethods, enabled, alwaysRun, invocationCount, threadPoolSize, timeOut (ms), expectedExceptions, dataProvider, retryAnalyzer, description.
>
> **Example:** timeOut = 30000 fails a checkout test that hangs; expectedExceptions for an invalid-quantity API test.

```
@Test(invocationCount = 3, timeOut = 30000, groups = "smoke")
public void checkout() { }
```

115. How do you run the same test method many times (e.g. 100 times with the same data)? **[asked x2 in your bank]**

> **Answer:** Use invocationCount (and threadPoolSize to run them in parallel).
>
> **Example:** Run the add-to-cart test 100 times to check stability or concurrency.

```
@Test(invocationCount = 100, threadPoolSize = 5)
public void addToCart() { }
```

116. How do you generate TestNG reports? **[asked x5 in your bank]**

> **Answer:** TestNG generates HTML reports in test-output (index.html, emailable-report.html); you can customize with IReporter or use Extent/Allure; Maven Surefire writes reports to target/surefire-reports.
>
> **Example:** Share emailable-report.html with the team after the nightly run.

Cucumber & BDD (10)

117. What are Background in Cucumber and how do you implement it? **[asked x4 in your bank]**

> **Answer:** Background holds steps common to every scenario of a feature; it runs before each scenario.
>
> **Example:** Every cart scenario starts with the user logged in.

```
Background:
  Given the user is logged in
  And the cart is empty
```

118. What are Data Tables? How do you convert them to a list or map? DataTable vs Examples? How do you handle test data in scenarios? **[asked x7 in your bank]**

> **Answer:** A Data Table passes a table of data into a single step; convert it with asList(), asMap() or asMaps(). Examples feed a Scenario Outline and repeat the whole scenario for each row.
>
> **Example:** A data table lists the products to add to the cart in one step.

```
@When("the user adds products")
public void add(DataTable t) { List<Map<String, String>> rows = t.asMaps(); }
```

119. How do you pass parameters in step definitions (regex, placeholders)? **[asked x4 in your bank]**

> **Answer:** Use Cucumber expressions such as {string}, {int}, {double} (or regex groups) in the step text and receive them as method parameters.
>
> **Example:** "When the user adds 2 of "Blue Shirt"" passes quantity and name.

```
@When("the user adds {int} of {string}")
public void add(int qty, String name) { }
```

120. How do you run Cucumber tests in parallel? **[asked x4 in your bank]**

> **Answer:** With cucumber-testng override the scenarios() DataProvider with parallel = true (or use JUnit Platform parallel settings) and keep a thread-safe driver using ThreadLocal and dependency injection.
>
> **Example:** Run login, cart and payment feature files at the same time.

```
@Override @DataProvider(parallel = true)
public Object[][] scenarios() { return super.scenarios(); }
```

121. How do you share data between steps (World/context object)? **[asked x3 in your bank]**

> **Answer:** Use a shared context (World) object injected with PicoContainer so step classes share driver and data per scenario.
>
> **Example:** The order ID created in a Given step is read in the Then step.

```
public class TestContext { public WebDriver driver; public String orderId; }
public class OrderSteps { OrderSteps(TestContext ctx) { this.ctx = ctx; } }
```

122. How do you generate Cucumber reports (HTML)? **[asked x3 in your bank]**

> **Answer:** Use plugins in the runner: html:, json:, pretty, or reporting libraries (Extent adapter, Masterthought) to produce HTML reports.
>
> **Example:** plugin = {"html:target/cucumber.html", "json:target/cucumber.json"}

123. How do you structure a Cucumber project and feature files? **[asked x5 in your bank]**

> **Answer:** Features grouped by module (cart, payment), step definitions by domain, separate hooks, runner, pages and utilities packages, plus resources for config and data.
>
> **Example:** src/test/resources/features/cart/*.feature and src/test/java/stepdefs/CartSteps.java.

124. How do you write reusable step definitions and follow best practices? **[asked x4 in your bank]**

> **Answer:** Write business-language parameterized steps, one purpose per step, no UI detail in feature files, organise steps by domain and avoid duplicates.
>
> **Example:** "When the user adds {string} to the cart" is reused by many scenarios.

125. What are the limitations of Cucumber?

> **Answer:** Extra layer to maintain, slower to write, risk of duplicated or UI-heavy steps and unreadable scenarios, and it only pays off with real business collaboration.
>
> **Example:** A purely technical team may find a Cucumber layer an overhead for simple regression.

126. What is the difference between BDD and TDD?

> **Answer:** TDD: developers write unit tests first (red-green-refactor). BDD: describes behaviour in business-readable scenarios and focuses on collaboration and acceptance.
>
> **Example:** TDD tests the discount function; BDD scenario describes "customer gets 10% off with a coupon".

Test Data, Configuration, Reporting & Logging (4)

127. How do you read JSON using ObjectMapper?

> **Answer:** Use Jackson ObjectMapper to read a JSON file into a POJO, Map or JsonNode.
>
> **Example:** Read test users from users.json.

```
ObjectMapper om = new ObjectMapper();
User u = om.readValue(new File("data/user.json"), User.class);
List<User> list = om.readValue(new File("data/users.json"),
    new TypeReference<List<User>>() {});
```

128. How do you ensure test data independence, isolate data for parallel runs and clean up after tests? **[asked x6 in your bank]**

> **Answer:** Create unique data per test (UUID/timestamp), prepare it through API, avoid shared records between parallel threads and clean up in @After methods.
>
> **Example:** Each test registers a new email like user_171234@test.com and deletes it afterwards.

129. How do you manage sensitive data like passwords and test credentials? **[asked x2 in your bank]**

> **Answer:** Never hard-code them: use environment variables, CI secrets or a vault, encrypt config values and mask them in logs and reports.
>
> **Example:** Jenkins credentials binding passes the payment test user password as an environment variable.

130. How do you implement logging in your framework (Log4j)? **[asked x3 in your bank]**

> **Answer:** Use Log4j2: a logger per class with levels (info, warn, error) and a file appender configured in log4j2.xml; log key steps and failures.
>
> **Example:** Log "Added SKU101 to cart" and the error on failure.

```
private static final Logger log = LogManager.getLogger(CartPage.class);
log.info("Added SKU101 to cart");
```

Cross-browser, Grid, Docker, CI/CD, Maven & Git (6)

131. What is Selenium Grid and RemoteWebDriver? How do you run tests on a Grid or remote machine? **[asked x6 in your bank]**

> **Answer:** Selenium Grid runs tests on remote machines/browsers in parallel. RemoteWebDriver connects to the Grid URL with the required browser options.
>
> **Example:** Run the suite on Chrome and Firefox nodes through one hub URL.

```
WebDriver driver = new RemoteWebDriver(new URL("http://localhost:4444"), new ChromeOptions());
```

132. How do you run Selenium tests in Docker or without a local browser? **[asked x3 in your bank]**

> **Answer:** Run the official selenium/standalone-chrome (or a docker-compose Grid) container and point RemoteWebDriver to it; no local browser is needed.
>
> **Example:** CI starts a Chrome container, runs tests against it and removes it.

```
docker run -d -p 4444:4444 --shm-size="2g" selenium/standalone-chrome
```

133. What is headless browser testing? Advantages, disadvantages and how to run it. **[asked x8 in your bank]**

> **Answer:** Headless runs the browser without a visible UI. Advantages: faster, CI friendly, lower resource use. Disadvantages: harder to debug and rendering/behaviour can differ. Enable with --headless=new.
>
> **Example:** Nightly regression runs headless on the Jenkins agent.

```
ChromeOptions o = new ChromeOptions();
o.addArguments("--headless=new", "--window-size=1920,1080");
```

134. What is Maven and how is it used? Structure of pom.xml, group id vs artifact id, running TestNG with Maven. **[asked x5 in your bank]**

> **Answer:** Maven manages build and dependencies. pom.xml has groupId (organisation), artifactId (project name), version, dependencies and plugins (Surefire for tests). Run TestNG with mvn test and the suite file.
>
> **Example:** Selenium, TestNG and Extent dependencies in the pom; mvn clean test runs the suite.

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <suiteXmlFiles><suiteXmlFile>testng.xml</suiteXmlFile></suiteXmlFiles>
  </configuration>
</plugin>
```

135. How do you use Git and version control in your automation project? **[asked x3 in your bank]**

> **Answer:** Use Git for branching, commits, pull requests and code reviews; ignore target and driver files; Jenkins checks out the repo for each run.
>
> **Example:** Feature branch per new page object, merged after review.

136. How do you handle a failed automation build in Jenkins?

> **Answer:** Open the console log, classify the failure (compile, environment, real test failure, flaky), reproduce, fix or raise a defect, rerun and notify the team.
>
> **Example:** Build failed due to an unreachable QA URL: report environment issue and rerun after recovery.

Scenario-Based & Real-World Automation (3)

137. How do you handle CAPTCHA in automation? **[asked x5 in your bank]**

> **Answer:** CAPTCHA is designed to block automation. Disable it or use a test key/bypass in the test environment, or keep that step manual.
>
> **Example:** Registration test uses a QA-only CAPTCHA bypass token.

138. How do you handle OTP in automation? **[asked x3 in your bank]**

> **Answer:** Use a fixed test OTP or bypass in the test environment, or read the OTP through an API/DB/test mailbox service; do not automate a real SMS.
>
> **Example:** QA environment accepts OTP 123456 for test numbers.

139. You are asked to automate a functionality that is not fully developed. How do you handle it? **[asked x2 in your bank]**

> **Answer:** Start with test design from acceptance criteria and mocks, build page objects once the UI stabilises, use stable test IDs, coordinate with developers and avoid heavy rework.
>
> **Example:** Prepare the Wishlist feature scripts from designs and finalise locators when the page is ready.

Java Concepts in the Selenium Framework (4)

140. How do you use interfaces and abstract classes in Selenium? Why are interfaces important in automation? **[asked x3 in your bank]**

> **Answer:** WebDriver itself is an interface. Use interfaces for contracts and abstract classes for shared page logic; they give loose coupling and make code swappable and testable.
>
> **Example:** abstract BasePage with waits; interface PaymentPage implemented by CardPage and UpiPage.

141. Which Java collections have you used in your framework? Do you use a property file, and which Java concept does it use? **[asked x4 in your bank]**

> **Answer:** List<WebElement>, Map for test data, Set for unique values. A property file uses the Properties class (file I/O with key-value pairs).
>
> **Example:** Properties for config; Map<String,String> for checkout form data.

142. How do you handle exceptions in Java Selenium scripts? Checked vs unchecked exceptions. **[asked x3 in your bank]**

> **Answer:** Use try-catch-finally for risky operations (file/DB, optional elements) and custom exceptions with clear messages. Checked exceptions must be handled; Selenium exceptions are unchecked (RuntimeException).
>
> **Example:** Wrap Excel reading in try-catch and throw a custom FrameworkException.

143. How are constructors, the this and super keywords, access modifiers and multiple classes used in the framework? **[asked x5 in your bank]**

> **Answer:** Page constructors accept the driver; this refers to the current object and super calls the parent constructor; locators private, actions public, shared methods protected in BasePage.
>
> **Example:** CartPage(WebDriver driver) { super(driver); }

```
public class CartPage extends BasePage {
  public CartPage(WebDriver driver) { super(driver); }
}
```

Automation Strategy, Metrics & Career (3)

144. What metrics do you track in test automation? How do you track coverage and calculate ROI? **[asked x4 in your bank]**

> **Answer:** Track automation coverage, pass rate, execution time, defects found, flaky rate and maintenance effort. ROI = (manual effort saved - automation cost) / automation cost.
>
> **Example:** Regression of 500 cases went from 3 days manual to 2 hours automated.

145. How do you estimate effort for automation?

> **Answer:** Break down by feature: analysis, page objects, scripts, data setup, debugging and review; consider complexity, test count, framework readiness and add a buffer.
>
> **Example:** Checkout module: about 25 tests, around 6-8 days including data setup and review.

146. Describe a time you owned the framework during a critical release. What are your key achievements in automation? **[asked x2 in your bank]**

> **Answer:** Use a STAR story: situation (critical release), task (own the framework), action (stabilised flaky tests, parallel runs, triage), result (regression time cut, release on time).
>
> **Example:** Before a festival sale release, cut regression from 5 hours to 50 minutes with parallel runs.

Section 3: Medium Priority (48 questions)

Occasionally asked, often as follow-ups or to probe a specific area from your resume. One clear answer each.

Selenium & WebDriver Fundamentals (9)

147. How do you open a URL, get the title, current URL and page source, navigate back/forward and refresh? **[asked x9 in your bank]**

> **Answer:** Use get(), getTitle(), getCurrentUrl(), getPageSource() and navigate().back()/forward()/refresh().
>
> **Example:** Open the shop home page and verify the title.

```
driver.get("https://shop.example.com");
String title = driver.getTitle();
driver.navigate().refresh();
```

148. How do you maximize, minimize and resize the browser window? What does driver.manage() return? **[asked x4 in your bank]**

> **Answer:** Use driver.manage().window(): maximize(), minimize(), setSize(Dimension). driver.manage() returns WebDriver.Options.
>
> **Example:** Resize to 375x812 to check the mobile layout.

```
driver.manage().window().maximize();
driver.manage().window().setSize(new Dimension(375, 812));
```

149. What is the difference between Selenium RC and WebDriver? **[asked x2 in your bank]**

> **Answer:** RC injected JavaScript through a server (slower, now removed); WebDriver talks directly to the browser through its driver using the W3C protocol.
>
> **Example:** Modern tests use WebDriver; RC no longer exists in Selenium 4.

150. What are the advantages of Selenium over other tools, and why do we still use it despite third-party jars? **[asked x2 in your bank]**

> **Answer:** Selenium is free, multi-language, multi-browser/OS and integrates with TestNG, Maven and CI; third-party jars (POI, Extent) fill reporting/data gaps, so it is still widely used.
>
> **Example:** Selenium plus Extent and POI forms a complete free solution.

151. Why is Java preferred for Selenium automation?

> **Answer:** Java is object-oriented, has a large community and libraries, strong tool support (Maven, TestNG) and is widely used with Selenium.
>
> **Example:** Java libraries read Excel test data and call REST APIs in the same framework.

152. What is WebDriverManager? **[asked x2 in your bank]**

> **Answer:** WebDriverManager downloads and sets up browser drivers automatically. Selenium 4.6+ includes Selenium Manager, so it is often not required.
>
> **Example:** new ChromeDriver() works without setting the driver path in recent versions.

153. What are DesiredCapabilities? **[asked x2 in your bank]**

> **Answer:** DesiredCapabilities set browser properties as key-value pairs. In Selenium 4 use browser Options classes (ChromeOptions) instead.
>
> **Example:** ChromeOptions for headless and incognito rather than DesiredCapabilities.

154. How do you verify the color, font and position of an element? **[asked x3 in your bank]**

> **Answer:** Use getCssValue(), getLocation(), getSize() or getRect(); convert colours with the Color class.
>
> **Example:** Check the Buy Now button is green and 16px.

```
String color = Color.fromString(btn.getCssValue("color")).asHex();
String size = btn.getCssValue("font-size");
Rectangle r = btn.getRect();
```

155. What is the difference between submit() and click()?

> **Answer:** submit() submits the enclosing form from any element inside it and works only for form elements; click() clicks the element and triggers its handlers like a user.
>
> **Example:** Prefer click() on Place Order because it behaves like the user.

Locators, XPath & CSS (2)

156. If proper XPath, CSS and ID are not available, how do you identify an element? What are the CSS selector attributes? **[asked x2 in your bank]**

> **Answer:** Use link text, relative XPath with text/sibling/ancestor, CSS attribute selectors ([attr*='x'], ^=, $=), JavaScript querySelector or ask developers for a test id. CSS selector attributes: #id, .class, [attr], [attr^='x'], [attr$='x'], [attr*='x'].
>
> **Example:** Button without id: //button[text()='Apply'] or div.coupon button.

157. With only CSS locators, how do you find the parent or grandparent of an element?

> **Answer:** Plain CSS cannot move to a parent; use XPath (.. or parent::) or the modern CSS :has() selector.
>
> **Example:** Find the product card (parent) containing a given price.
>
> ```
> By.xpath("//span[text()='999']/ancestor::div[@class='product']")
> By.cssSelector("div.product:has(span.price)")
> ```

Waits & Synchronization (2)

158. Does a script work without an implicit wait?

> **Answer:** Yes. Implicit wait is optional (default 0). Scripts work when elements are ready or when explicit waits are used.
>
> **Example:** The suite uses only explicit waits.

159. How do you define page load strategies? What is Page Load Timeout? **[asked x2 in your bank]**

> **Answer:** Page load strategy: normal (default, waits for full load), eager (DOMContentLoaded) or none. Page load timeout is the maximum time for a page to load before a TimeoutException.
>
> **Example:** Use eager on a heavy product page.
>
> ```
> o.setPageLoadStrategy(PageLoadStrategy.EAGER);
> driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));
> ```

Element Interactions, Actions & JavaScriptExecutor (9)

160. When should you use sendKeys() vs sendKeys() from the Actions class? **[asked x2 in your bank]**

> **Answer:** WebElement.sendKeys types into that element. Actions.sendKeys works on the focused element and can be chained with other actions or key combinations.
>
> **Example:** Actions chain to type text and press TAB.
>
> ```
> new Actions(driver).sendKeys(Keys.TAB).sendKeys("Pune").perform();
> ```

161. How do you handle checkboxes and radio buttons (e.g. click all checkboxes)? **[asked x2 in your bank]**

> **Answer:** Check isSelected() before clicking; for all checkboxes use findElements and loop. Radio buttons are selected with click().
>
> **Example:** Select all product filters or choose a payment method radio button.
>
> ```
> for (WebElement c : driver.findElements(By.cssSelector("input[type='checkbox']")))
>   if (!c.isSelected()) c.click();
> ```

162. How do you validate infinite scroll and lazy-loaded content? **[asked x2 in your bank]**

> **Answer:** Scroll with JavaScript in a loop until the item count stops increasing, then assert the new items loaded.
>
> **Example:** Scroll the product list until all 100 items load.

```
int before;
do {
  before = driver.findElements(By.cssSelector(".product")).size();
  js.executeScript("window.scrollTo(0, document.body.scrollHeight);");
  Thread.sleep(1000);   // replace with a wait on the item count in real code
} while (driver.findElements(By.cssSelector(".product")).size() > before);
```

163. How do you handle cookies? **[asked x2 in your bank]**

> **Answer:** Use driver.manage(): getCookies(), getCookieNamed(), addCookie(), deleteCookieNamed(), deleteAllCookies().
>
> **Example:** Save the login cookie and reuse it to skip the login UI.

```
Cookie session = driver.manage().getCookieNamed("session");
driver.manage().addCookie(new Cookie("session", session.getValue()));
```

164. How do you verify tooltips? **[asked x2 in your bank]**

> **Answer:** Hover over the element with Actions and read the title attribute or the tooltip element's text.
>
> **Example:** Hover the info icon next to Delivery and check the tooltip text.

```
new Actions(driver).moveToElement(icon).perform();
String tip = wait.until(ExpectedConditions
    .visibilityOfElementLocated(By.cssSelector(".tooltip"))).getText();
```

165. How do you capture network traffic, simulate network conditions and read browser console logs? **[asked x4 in your bank]**

> **Answer:** Selenium 4 DevTools (Chromium) can listen to network calls, emulate network conditions and read console logs.
>
> **Example:** Check the add-to-cart API call returns 200 and no console errors.

```
DevTools dt = ((HasDevTools) driver).getDevTools();
dt.createSession();
dt.send(Network.enable(Optional.empty(), Optional.empty(), Optional.empty()));
dt.addListener(Network.responseReceived(), r ->
    System.out.println(r.getResponse().getUrl() + " " + r.getResponse().getStatus()));
```

166. How do you get text from a text box when getText() does not work?

> **Answer:** For input fields the value is a property, not text, so use getDomProperty("value") (or getAttribute("value")).
>
> **Example:** Read the quantity typed into the cart quantity box.

```
String qty = driver.findElement(By.id("qty")).getDomProperty("value");
```

167. How do you get dropdown values and print them in ascending order (TreeSet)? **[asked x2 in your bank]**

> **Answer:** Get the options with Select.getOptions(), put the text in a TreeSet (sorted) and print it.
>
> **Example:** Verify the sort-by options list in ascending order.

```
Set<String> sorted = new TreeSet<>();
for (WebElement o : new Select(dropdown).getOptions()) sorted.add(o.getText());
System.out.println(sorted);
```

168. How do you take element-only and full-page screenshots? **[asked x2 in your bank]**

> **Answer:** Element screenshot: WebElement.getScreenshotAs(). Full page: Firefox getFullPageScreenshotAs(), or Chrome DevTools/AShot for full page.
>
> **Example:** Capture only the order summary block, and a full page for the report.

```
File f = summary.getScreenshotAs(OutputType.FILE);
```

Windows, Frames, Alerts & Pop-ups (4)

169. If you do not accept an alert, what happens?

> **Answer:** While an alert is open other commands fail with UnhandledAlertException, so you must accept or dismiss it (or set an unhandled prompt behavior).
>
> **Example:** Leave the delete confirmation open and the next click throws an exception.

170. How do you open a new blank window/tab, close only the current tab or close all tabs? **[asked x3 in your bank]**

> **Answer:** Open with switchTo().newWindow(WindowType.TAB); close() closes the current tab, quit() closes all.
>
> **Example:** Open the order tracking page in a new tab and close just that tab.

```
driver.switchTo().newWindow(WindowType.TAB);
driver.get("https://shop.example.com/track");
driver.close();   // closes only this tab
```

171. What is the difference between frames and iframes and how are they identified? **[asked x2 in your bank]**

> **Answer:** frame belongs to an old frameset layout; iframe embeds another page inside the page. Both are handled with switchTo().frame() and found by tag name, id, name or index.
>
> **Example:** Payment form in an iframe is accessed by id.

172. How do you test scenarios with third-party iFrames?

> **Answer:** Switch into the iframe, wait for its content, use sandbox/test accounts for the third party, and limit assertions to what you control; mock unstable third-party content.
>
> **Example:** Test the payment iframe only up to the payment-provider boundary in the sandbox.

Exceptions & Troubleshooting (1)

173. What if the execution aborts mid-suite due to an unhandled exception?

> **Answer:** In TestNG a failing test does not abort the suite, but a failure in @BeforeSuite/@BeforeClass skips dependent tests. Use try-catch/finally in utilities and @AfterMethod(alwaysRun = true) for cleanup, and a listener to log.
>
> **Example:** Make the browser quit even when the test throws an exception.

```
@AfterMethod(alwaysRun = true)
public void tearDown() { if (driver != null) driver.quit(); }
```

Flaky Tests, Retry & Debugging (1)

174. What is self-healing in automation and is it possible with Selenium?

> **Answer:** Self-healing finds a new locator automatically when the old one fails. Selenium has no built-in support; you can add fallback locators in a wrapper or use tools such as Healenium or commercial AI tools.
>
> **Example:** If #add-to-cart changes, a fallback locator by text finds the button and logs a warning.

File Upload/Download & Broken Links (1)

175. How do you count the links on a page (by tag) and handle hidden links? **[asked x2 in your bank]**

> **Answer:** Use findElements(By.tagName("a")).size(); for hidden links filter with isDisplayed() or read them through JavaScript.
>
> **Example:** Count total and visible links in the footer.

```
List<WebElement> links = driver.findElements(By.tagName("a"));
long visible = links.stream().filter(WebElement::isDisplayed).count();
```

Framework Design & Architecture (4)

176. What are the SOLID principles and how do you apply them in automation? **[asked x2 in your bank]**

> **Answer:** S: one responsibility per page class; O: extend BasePage instead of modifying it; L: child pages usable as parents; I: small interfaces; D: depend on WebDriver, not ChromeDriver.
>
> **Example:** CartPage only handles cart actions; tests depend on WebDriver.

177. What is the difference between a framework and a library? **[asked x2 in your bank]**

> **Answer:** A library is code you call (Selenium); a framework provides the structure and calls your code (TestNG, Cucumber). Your automation framework combines libraries with a structure.
>
> **Example:** Selenium is the library; TestNG runs your tests inside its structure.

178. How do you implement error handling and recovery in the framework? **[asked x2 in your bank]**

> **Answer:** Use try-catch in utilities with clear messages, custom exceptions, screenshots/logs on failure, retry for flaky steps and cleanup in finally/@AfterMethod.
>
> **Example:** If the browser crashes mid-test, restart the driver and rerun the test once.

179. How do you implement custom assertions and validations?

> **Answer:** Create wrapper methods that assert, log to the report and give clear messages (soft or hard), for domain checks such as prices.
>
> **Example:** assertPriceEquals(actual, expected) logs both values to the report.

```
public static void assertPrice(double actual, double expected) {
  Reporter.log("Price actual=" + actual + " expected=" + expected);
  Assert.assertEquals(actual, expected, 0.01, "Price mismatch");
}
```

TestNG (1)

180. What is the @Factory annotation? Factory vs DataProvider (with a real-time example)? **[asked x3 in your bank]**

> **Answer:** @Factory creates several instances of a test class, each with different constructor data. DataProvider runs one method repeatedly with different data. Factory suits class-level variation (browser/user per instance).
>
> **Example:** One class instance per user role, each with its own login state.

```
@Factory
public Object[] create() {
  return new Object[] { new LoginTest("admin"), new LoginTest("user") };
}
```

Cucumber & BDD (4)

181. What are common challenges in writing Cucumber tests and how do you keep scenarios readable and maintainable? **[asked x2 in your bank]**

> **Answer:** Challenges: long or UI-focused scenarios, duplicated steps, tight coupling to UI. Keep scenarios short, declarative, in business language and review them with the product owner.
>
> **Example:** Write "When the user pays by card" instead of ten click steps.

182. How do you handle dynamic locators in Cucumber steps?

> **Answer:** Pass the value from the step as a parameter and build the locator with String.format.
>
> **Example:** A step selects any product by name.

```
By product = By.xpath(String.format("//h2[text()='%s']", name));
```

183. Can you share an example of a complex scenario you wrote? How do you ensure collaboration between developers, testers and business in BDD? **[asked x2 in your bank]**

> **Answer:** Example: "Place an order using a coupon, wallet balance and COD" scenario with a data table. Collaboration: three-amigos sessions, shared feature files in Git and review of scenarios before development.
>
> **Example:** Business, developer and tester agree on the coupon rules before coding.

184. How do you skip scenarios in Cucumber? How do you integrate Cucumber in a Maven project? **[asked x3 in your bank]**

> **Answer:** Skip with a tag such as @wip and run with tags = "not @wip". For Maven add cucumber-java and cucumber-testng dependencies and run mvn test, optionally with -Dcucumber.filter.tags.
>
> **Example:** Mark an unfinished wishlist scenario @wip so the build ignores it.

```
mvn test -Dcucumber.filter.tags="@smoke and not @wip"
```

Scenario-Based & Real-World Automation (5)

185. You are asked to automate a legacy application. What is your approach? **[asked x2 in your bank]**

> **Answer:** Understand the app technology and risks, check locator stability, automate the high-value stable flows first, use waits and JavaScript where needed, and use another tool if Selenium cannot reach it.
>
> **Example:** Automate login and order entry in an old back-office portal.

186. How would you design tests for a shopping cart and a multi-step form with validation? **[asked x2 in your bank]**

> **Answer:** Cart: add/remove/update quantity, price totals, coupon, persistence after login. Multi-step form: mandatory/format validations, boundary values, back/next data retention, error messages. Drive with data and page objects per step.
>
> **Example:** Checkout address step: blank pincode shows an error and the Next button stays disabled.

187. How do you validate an email sent after registration or an email verification link? **[asked x3 in your bank]**

> **Answer:** Use a test mailbox (Mailosaur, MailHog, Mailtrap) or an email API, fetch the mail by subject, extract the verification link and open it with the driver.
>
> **Example:** Registration test reads the verification link and opens it.

```
String link = mailApi.getLatestMail(email).extractLink("verify");
driver.get(link);
```

188. How do you test responsive layouts and mobile web with Selenium? **[asked x4 in your bank]**

> **Answer:** Use ChromeOptions mobile emulation or setSize for responsive layouts and check element visibility; real devices need Appium or a cloud grid.
>
> **Example:** Verify the hamburger menu appears at 375 px width.

```
Map<String, String> m = Map.of("deviceName", "iPhone 12");
ChromeOptions o = new ChromeOptions();
o.setExperimentalOption("mobileEmulation", m);
```

189. How do you measure page load time through automation? **[asked x2 in your bank]**

> **Answer:** Read the Navigation Timing API with JavaScript or measure the time around the page load; use Lighthouse for detailed performance.
>
> **Example:** Check the product page loads in under 3 seconds.

```
long load = (Long) js.executeScript(
    "return performance.timing.loadEventEnd - performance.timing.navigationStart;");
```

Automation Strategy, Metrics & Career (5)

190. What is the risk of over-automation? How do you avoid duplicate and redundant tests? **[asked x3 in your bank]**

> **Answer:** Over-automation creates high maintenance, slow suites and false failures. Avoid duplicates through test design reviews and traceability; automate by risk and value.
>
> **Example:** Do not automate a promo banner that changes every week.

191. Why avoid hardcoded values, and how do you ensure reusability in test methods? **[asked x2 in your bank]**

> **Answer:** Hard-coded values make scripts brittle and hard to reuse; use config, data files, constants and parameterized methods.
>
> **Example:** Read the base URL and credentials from config instead of the test.

192. How should you handle test dependencies and execution order? Why should tests be atomic? **[asked x3 in your bank]**

> **Answer:** Tests should be atomic and independent so they can run in any order and in parallel; create data through API/setup; use dependsOnMethods only sparingly.
>
> **Example:** Each test creates its own cart instead of reusing the previous test's cart.

193. How often should you refactor automation scripts?

> **Answer:** Refactor continuously: when touching code, at the end of each sprint/release, and whenever you see duplication or flaky patterns.
>
> **Example:** Merge duplicate login steps into one reusable method.

194. How do you implement validation for negative test cases?

> **Answer:** Use invalid, boundary and missing data, assert the exact error messages and that no state changed; drive with DataProvider.
>
> **Example:** Coupon "EXPIRED10" must show an error and leave the total unchanged.
>
> ```
> @DataProvider public Object[][] badCoupons() { return new Object[][] {{"EXPIRED10"}, {""}}; }
> ```

Section 4: Low Priority / Quick Revision (11 questions)

Niche or rarely asked at 4+ years. Skim them so you are never caught off guard.

Selenium & WebDriver Fundamentals (1)

195. Why do we use getAriaRole() and getAccessibleName()?

> **Answer:** Selenium 4 methods that return an element's ARIA role and accessible name; used for accessibility checks and semantic locators.
>
> **Example:** Check the Add to Cart button's role is button and its accessible name is "Add to Cart".

```
String role = btn.getAriaRole();
String name = btn.getAccessibleName();
```

Element Interactions, Actions & JavaScriptExecutor (3)

196. How do you highlight an element before clicking (JavaScript)? **[asked x3 in your bank]**

> **Answer:** Use JavascriptExecutor to change the element's style border before clicking, which helps debugging and demos.
>
> **Example:** Highlight the Place Order button in the screenshot.

```
js.executeScript("arguments[0].style.border='3px solid red';", element);
```

197. How do you handle browser notifications?

> **Answer:** Disable them with browser options or preferences (--disable-notifications).
>
> **Example:** Block the "Allow notifications" prompt on the shop home page.

```
o.addArguments("--disable-notifications");
```

198. How would you cover character keyboard operation from the context menu using a user-defined keyword? (legacy tool question)

> **Answer:** In Selenium, a keyword is a reusable method. Open the context menu with Actions.contextClick and press keys with Keys.ARROW_DOWN and ENTER.
>
> **Example:** Right-click a product image and choose an option using the keyboard.

```
new Actions(driver).contextClick(el).sendKeys(Keys.ARROW_DOWN, Keys.ENTER).perform();
```

TestNG (1)

199. What are custom annotations in TestNG, and what is the role of annotations in framework optimization? Can test behavior be changed at runtime? **[asked x3 in your bank]**

> **Answer:** Custom annotations are created with @interface and read through reflection or a listener. IAnnotationTransformer can modify @Test attributes at runtime (for example, apply a retry analyzer to all tests).
>
> **Example:** Apply a retry analyzer to every test without editing each class.

```
public class RetryTransformer implements IAnnotationTransformer {
  public void transform(ITestAnnotation a, Class c, Constructor k, Method m) {
    a.setRetryAnalyzer(Retry.class);
  }
}
```

Cucumber & BDD (1)

200. What is the significance of the cucumber.properties or cucumber.yml file?

> **Answer:** It stores Cucumber configuration (glue, plugin, tags) outside the runner class. Modern Cucumber uses cucumber.properties (or junit-platform.properties); cucumber.yml is not used in current versions.
>
> **Example:** Keep the report plugin and tags in cucumber.properties.

```
cucumber.glue=stepdefs
cucumber.plugin=pretty, html:target/cucumber.html
cucumber.filter.tags=@smoke
```

Scenario-Based & Real-World Automation (1)

201. How do you automate localization testing (different UI languages)?

> **Answer:** Pass the locale to the browser (--lang), keep expected texts per language in resource files and assert labels, dates and currency for each language.
>
> **Example:** Check the checkout page labels in English and Hindi.

```
o.addArguments("--lang=hi");
```

Automation Strategy, Metrics & Career (4)

202. How do you conduct automation code reviews and walkthroughs, document the framework and version test cases? **[asked x3 in your bank]**

> **Answer:** Use pull-request reviews with a checklist, walkthrough sessions, a README or wiki for the framework, and Git branches/tags to version tests.
>
> **Example:** Every new page object is reviewed before merging.

203. What is the role of the traceability matrix in QA?

> **Answer:** The RTM maps requirements to test cases and automation scripts to prove coverage and support impact analysis.
>
> **Example:** Requirement "apply coupon" is linked to manual cases and the automated script CouponTest.

204. What are your daily activities as an automation engineer? How do you stay current and balance manual and automation work? **[asked x3 in your bank]**

> **Answer:** Daily: stand-up, analyse nightly failures, fix flaky tests, automate sprint stories, review code and maintain the framework. Stay current with blogs, courses and release notes and balance manual and automation by priority.
>
> **Example:** Morning: triage the nightly regression; afternoon: automate new wishlist stories.

205. How do you test the stability, data reusability, performance and rollback of the automation framework? **[asked x4 in your bank]**

> **Answer:** Run the suite several times to measure flaky rate, check data can be reused, track execution time trends, keep Git tags to roll back and run smoke on every framework change.
>
> **Example:** A framework change is validated with a smoke run on a branch before merging.